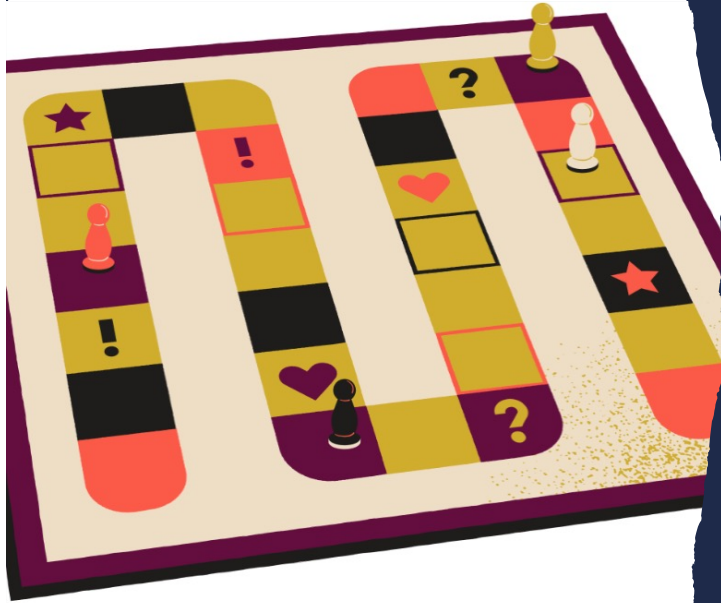




Du jeu console au jeu web

Architecture client-serveur, MVC, et NiceGUI

Pourquoi une GUI dans le navigateur ?

Vos besoins

- Plusieurs joueurs, parfois sur des machines différentes
- Cacher l'information : chacun voit ses cartes, pas celles des autres
- Pour le Loup-garou : partie partagée par un code (PIN)

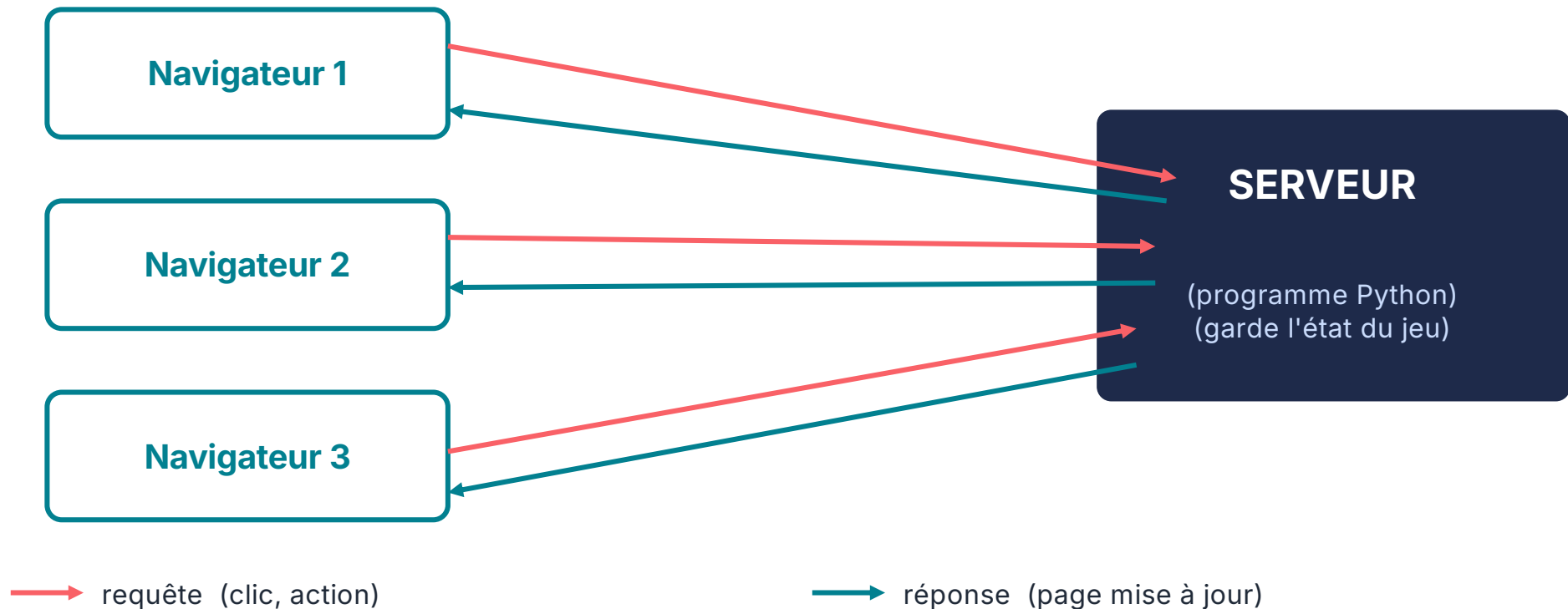
Une interface web résout tout ça

- Le serveur garde le « cerveau » du jeu
- Chaque joueur ouvre simplement une page
- Le serveur décide qui voit quoi
- Mise en ligne facile (plus tard)

→ On va utiliser NiceGUI : on écrit du Python, ça produit une page web.

Architecture client-serveur

Le client demande, le serveur répond.



Concrètement, dans vos jeux

Poker

Le serveur connaît toutes les cartes. À chaque joueur il n'envoie QUE sa main.

Codenames

Les espions voient la grille des couleurs. Les agents voient seulement les mots.

Loup-garou

Une partie = un PIN. Le serveur garde plusieurs parties en parallèle. Les loups-garous voient les autres loups-garous.

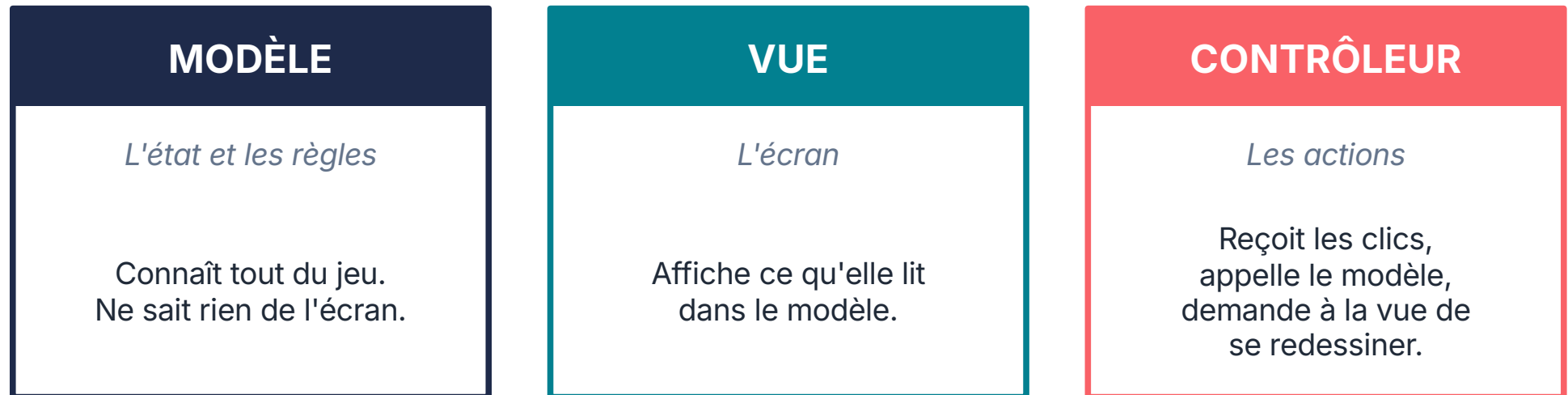
Carcassonne

Pas de cachette d'information, mais le tour de jeu est partagé entre joueurs.

Dans tous les cas : le serveur sait tout, mais n'envoie à chaque navigateur que ce qu'il a le droit de voir.

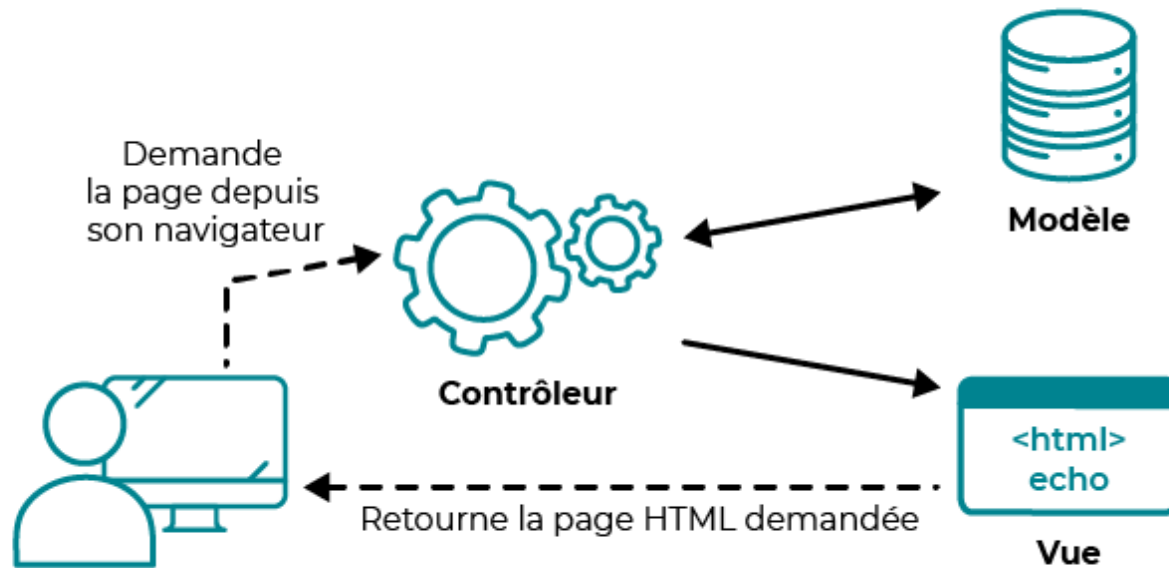
Organiser le code : le pattern MVC

Trois rôles distincts dans une application interactive.



Avantage : on peut changer la vue (texte → web → mobile) sans toucher aux règles du jeu.

Organiser le code : le pattern MVC



MVC dans vos projets

MODÈLE

classes.py

```
class Partie  
class Joueur  
les règles, l'état
```

VUE

app.py — affichage

```
ui.label(...)  
ui.button(...)  
ui.card(...)
```

CONTRÔLEUR

app.py — réactions

```
partie.faire_choix(...)  
afficher.refresh()
```

Votre version texte avait tout mélangé. Maintenant on sépare proprement.

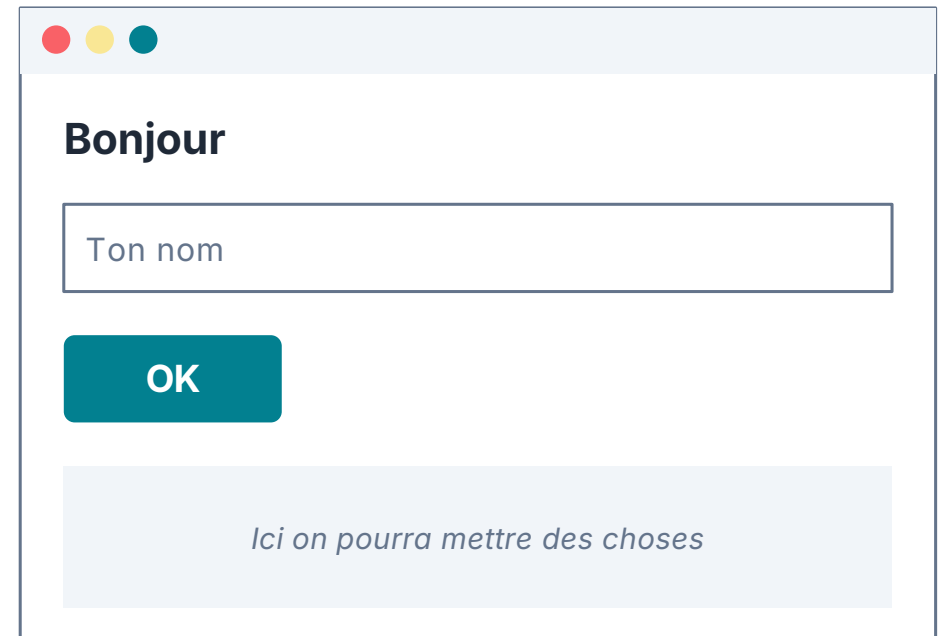
NiceGUI — les widgets

Un widget = un élément graphique. On en place sur la page comme des Lego.

Code Python

```
ui.label("Bonjour")  
ui.input("Ton nom")  
ui.button("OK")  
ui.card() # crée un espace vide, gris
```

Ce qui apparaît à l'écran



NiceGUI — événements & mise à jour

Un widget peut réagir à une action utilisateur (clic, frappe...). C'est le contrôleur.

```
def action():  
    partie.faire_choix("Alice", True)    # ← appel au modèle  
    afficher.refresh()                  # ← redessin de la vue  
  
ui.button("Parier", on_click=action)
```



Le navigateur ne calcule rien. Il affiche ce que le serveur lui envoie après refresh.

À nous de jouer !

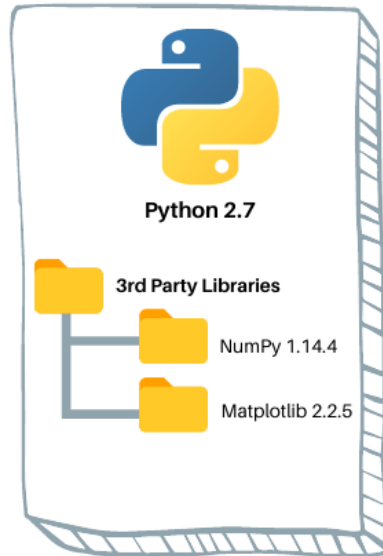
Petit jeu d'exemple : pari sur la carte la plus haute

- Version 1 : une seule fenêtre — on regarde le pattern MVC en action
- Version 2 : une fenêtre par joueur — l'architecture client-serveur en vrai
- Bonus : tout le monde se connecte sur ma machine, on joue ensemble

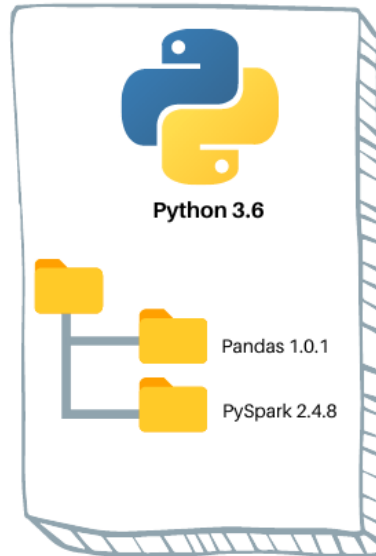
C'est parti avec NiceGUI !

Tout d'abord créons un v-env Python

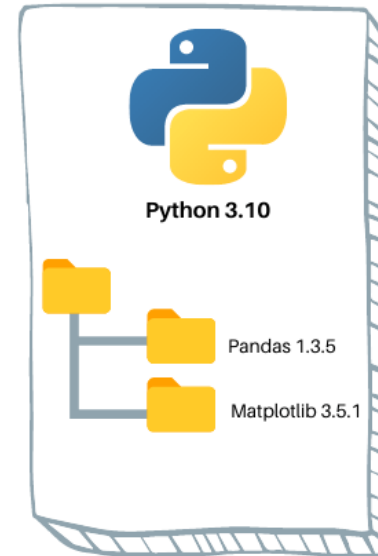
Virtual Environment 1



Virtual Environment 2



Virtual Environment 3



Installation — macOS

1. Ouvrir le terminal de VS Code
2. Créer l'environnement virtuel et l'activer

```
python3 -m venv .venv  
source .venv/bin/activate
```

3. Installer NiceGUI dans le venv

```
python -m pip install nicegui
```

Le prompt doit afficher (.venv) au début. Si non : refaire le « source ».

Installation — Windows

1. Ouvrir le terminal de VS Code

2. Créer l'environnement virtuel et l'activer

```
python -m venv .venv  
.venv\Scripts\activate
```

3. Installer NiceGUI dans le venv

```
python -m pip install nicegui
```

Si l'activation est bloquée : Set-ExecutionPolicy -Scope CurrentUser RemoteSigned

VS Code : sélectionner l'interpréteur du venv

Pour que le bouton ► Run utilise le bon Python (celui du venv).

- ① Cmd / Ctrl + Shift + P
- ② Taper : Python: Select Interpreter
- ③ Choisir l'option avec `.venv` (généralement marquée « Recommended »)

Aperçu de la liste qui apparaît :

Python 3.14	<code>.venv/bin/python</code>	← Recommended
Python 3.14	<code>/opt/homebrew/bin/python3</code>	Global

Vérification : en bas à droite de VS Code, on doit voir « `.venv (3.14.x)` ».

No more `int(input(...))` !

Le programme interroge l'utilisateur

```
class Joueur:
    def __init__(self, nom):
        self.nom = nom

class Partie:

    def __init__(self):
        self.joueurs = []

    def setup_players(self):
        np = int(input("Combien de joueurs ? "))
        for i in range(np):
            nom = input(f"Nom du joueur {i+1} : ")
            self.joueurs.append(Joueur(nom))
```

L'utilisateur interroge le programme

modèle

```
class Joueur:
    def __init__(self, nom):
        self.nom = nom

class Partie:
    def __init__(self, noms_joueurs):
        self.joueurs = [Joueur(nom) for nom in noms_joueurs]
```

vue + control.

```
@ui.page('/')
def accueil():
    ui.label("Configuration de la partie").style('font-size: 1.5em')
    nom1 = ui.input('Nom joueur 1', value='Alice')
    nom2 = ui.input('Nom joueur 2', value='Bob')

    def commencer():
        global partie
        partie = Partie([nom1.value, nom2.value])
```